

教務處要改系統，哪些資料不能變？

自編案例：導師查 DB101 資料庫課程成績，註冊組管理學生資料。規則：StudentId 學號必填且唯一；Name 姓名可重複；每筆成績指向已登記學生。



線表示本例資料對照，不是基數或繼承符號。

完整初始資料

資料	欄位與目前內容
STUDENT	(StudentId, Name) : S1 Alex ; S2 Blair
RESULT	(StudentId, CourseCode, Grade) : S1 DB101 80 ; S2 DB101 60

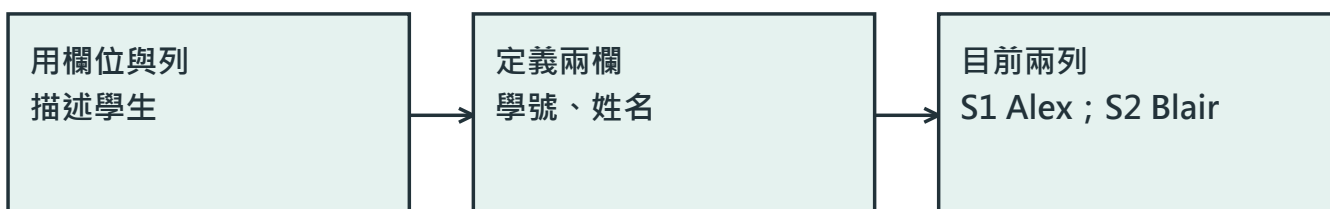
問題：導師只看到 Alex 與 80，表示系統不用學號？

回答：不是。S1 將姓名與成績連在一起；畫面省略學號，不代表整體資料沒有它。

本頁重點：先固定資料、規則與使用者需求，再判斷系統改了哪一層。

用表格描述、訂好欄位、填入學生，是三件事

註冊組仍保存 S1 Alex、S2 Blair。學號是 StudentId，姓名是 Name；規則仍是學號必填且唯一、姓名可重複。先看描述方式、欄位定義與目前值的差別。



箭頭連起描述方式、這張表的定義及目前內容，不是三次新增學生。

用圖中的資料說明名稱

圖中所指	名稱與意思
用表格的欄位與列描述資料	Relational model：提供這些描述概念
STUDENT(StudentId, Name)；學號必填且唯一	Schema：這份資料的結構與限制
目前是 S1 Alex、S2 Blair	State：目前資料；(S2, Blair) 只是其中一行

問題：新增 S3 Casey，就從 relational model 變成另一種 model？

回答：沒有。仍用相同欄位與列表表示學生，只是 state 多一行。

本頁重點：Model 提供描述概念；schema 是這份資料的定義；state 是目前內容。

新增 Casey，還是新增 Phone？

兩種修改都從 STUDENT 的 S1 Alex、S2 Blair 開始，互不接續。Phone 是電話；B 明訂允許尚未提供，用 NULL 表示。

A：新增一位學生

欄位：StudentId、Name

S1 Alex；S2 Blair

新增 S3 Casey

B：新增 Phone 欄位

欄位：StudentId、Name、Phone

S1 Alex NULL

S2 Blair NULL

A 增加一行；B 增加每行可記錄的一個欄位，並保留原兩位學生。

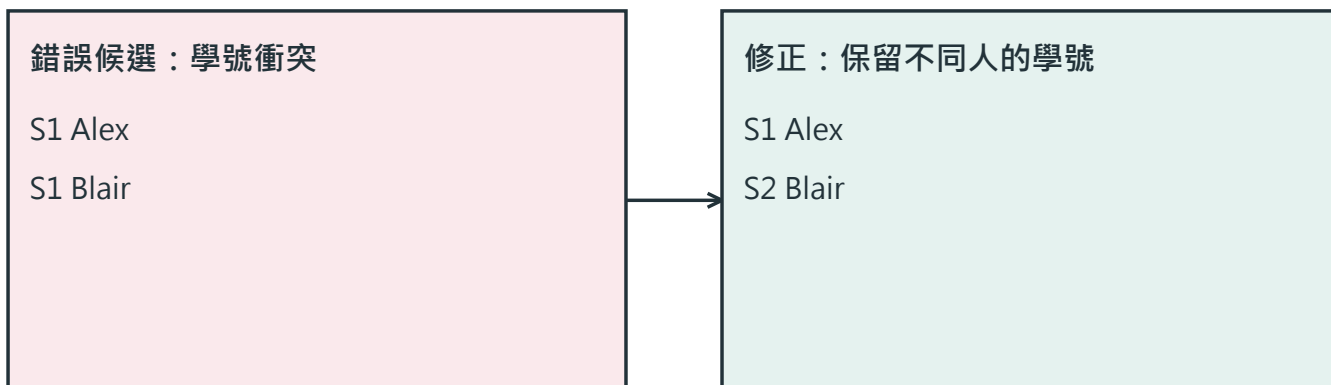
問題：B 出現 NULL，就表示原來沒有 S1 嗎？

回答：不是。S1 Alex 仍在，只是沒有電話值；「沒有一行」與「一行中的值尚未提供」不同。

本頁重點：A 改 state；B 改 schema，且要處理既有行的新欄位值。

姓名可重複，學號不可以

固定規則：StudentId 必填且唯一，Name 可重複。以下各候選狀態都獨立比較，不把上一頁新增資料帶入。



兩個人不能共用 S1；修正採原本已確認的 S2，不任意刪掉 Blair。

其他候選 state

資料	依固定規則判斷
S1 Alex；新生 S9 Alex	合法：另一位新生可同名，不更改 S2 Blair
沒有任何學生列	未違反本例唯一規則
NULL Alex	錯誤：學號必填

問題：初始名單沒有同名者，能推論姓名永遠不能重複？

回答：不能。是否允許來自規則，不是這兩筆資料剛好長什麼樣。

本頁重點：合法 state 必須符合已訂 schema，不能從小樣本猜永久規則。

導師畫面和整體資料，不是同一層

導師只看 S1 Alex 在 DB101 的 80 分。整體資料仍含 S2 Blair，以及兩張表之間的學號對照。



由上到下是 external、conceptual、internal schema；層間對應稱 mapping。三框描述同一資料庫，不是三份副本；位置只是示意。

問題：錯誤：導師畫面沒顯示 StudentId，所以可以把學號刪掉。

回答：不行。整體結構仍用學號對照學生與成績。修正是保留 S1、S2，只在導師畫面選擇顯示欄位。

本頁重點：Three-schema architecture 區分使用者描述、整體描述與儲存描述。

加索引：答案應不變，不保證更快

查詢固定為「找姓名 Blair 的學號」。資料固定 S1 Alex、S2 Blair；只增加 Name 的索引。索引是額外查找結構，不是新學生。

A：尚未新增 Name 索引

需求：Name = Blair

結果：S2

B：新增 Name 索引後

需求仍是 Name = Blair

結果仍是 S2

同一查詢在 SQLite 前後執行皆得 S2；沒有量測速度或保證採用哪一個計畫。

錯誤主張與修正

主張	修正依據
加索引後應只剩 Blair 那一列	STUDENT 仍有 Alex、Blair；索引不是刪除資料
結果相同，就證明變快	同樣得到 S2 只證明這次答案相同

問題：索引改了什麼，什麼必須保留？

回答：改的是 internal 存取結構；學生欄位、兩位學生與原查詢意義不變。這種隔離稱 physical data independence。

本頁重點：隔離實體儲存變動，與是否提升速度，是不同問題。

新增電話，導師的舊畫面能保留嗎？

舊畫面只讀學號、姓名。兩個分支各從 S1 Alex、S2 Blair 開始；Phone 電話不是這個畫面必須顯示的資料。

A：新增可空白的 Phone

仍保留 StudentId、Name

舊畫面讀原兩欄

仍得 S1 Alex；S2 Blair

B：刪除且不再保存 Name

學生只剩學號

要求顯示原姓名

已無資料可提供姓名

A 可用適當 mapping 維持 view；B 把所需事實刪掉，不能靠 data independence 補回。

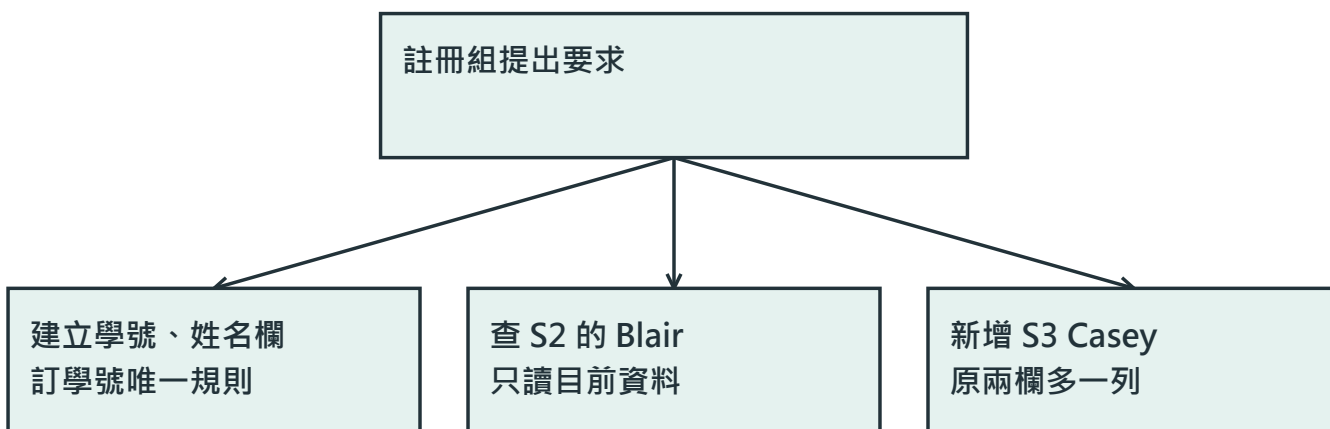
問題：修正 B 是否只要把畫面標題改成 Name？

回答：不夠。必須保留姓名事實及其對應，或明確變更使用者需求，不能只改標籤。

本頁重點：Logical data independence 要維持外部需求，不能把任意資訊損失都說成相容。

建欄位、查學生、新增學生，各改什麼？

註冊組要管理相同 STUDENT。以下每項都是獨立要求；本頁先辨認用途，不要求記 SQL 語法。



建立欄位與限制用 data definition language (DDL) ；讀取或增刪修改資料用 data manipulation language (DML) 。此處不教語法。

錯誤分類

錯誤說法	修正
按表單新增 S3 沒寫 SQL，所以沒操作資料庫	表單程式仍送出資料操作
查 Blair 是新增資料	讀取只回傳目前已有的 Blair

問題：查 Blair 時，必須自行指定磁碟位置嗎？

回答：不必。可以只表達要哪些資料，讓 DBMS 決定存取方式；這種表達方式稱高階查詢。

本頁重點：用途看改定義或操作資料，不看使用者有沒有手打指令。

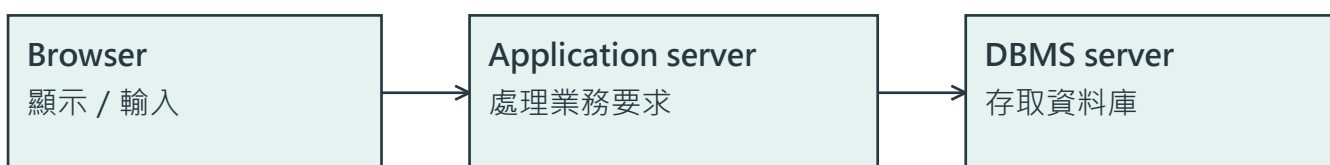
查同一份成績，請求經過哪些軟體？

學校要讓學生查自己的成績。A 使用直接連 DBMS 的校務應用程式；B 改成瀏覽器經應用伺服器。兩種方式都使用相同學生與成績。

A：應用程式直接請 DBMS 查資料



B：瀏覽器先把請求交給應用伺服器



A 的用戶端、DBMS 兩層稱 two-tier；B 多一層應用伺服器，稱 three-tier。箭頭是請求路徑；軟體職責不等於電腦台數。

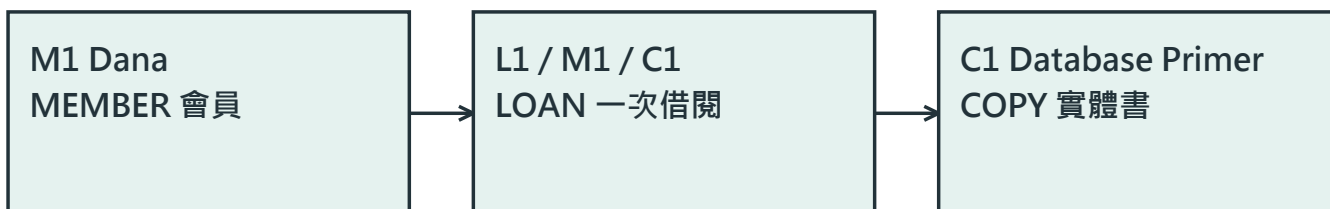
問題：B 的三個框，就等於 external、conceptual、internal？

回答：不是。B 多的是處理業務的應用伺服器；three-schema 描述同一資料庫的三個層次。修正讀法是沿箭頭追查請求，不把軟體當成資料描述。

本頁重點：先看框在表示軟體職責，還是在描述資料層次。

圖書館要加自助查詢，先看原資料

新案例：館員登記借閱，會員用自助畫面查書。MemberId 會員編號必填且唯一，姓名可重複；LoanId 識別一次借閱，每筆須指向已登記會員與實體書。



資料對照：L1 連到 M1 與 C1。M2 是另一位會員，尚未借書。

完整初始資料

表格與欄位	目前列
MEMBER(MemberId, Name)	M1 Dana ; M2 Evan
COPY(CopyId, Title)	C1 Database Primer
LOAN(LoanId, MemberId, CopyId)	L1 M1 C1

問題：Evan 的畫面是空的，是否代表沒有 LOAN 表？

回答：不是。LOAN 的結構存在，也有 Dana 的 L1；Evan 只是沒有自己的借閱列。

本頁重點：結構存在、表中有資料、某位會員有配對，是三件不同的事。

新增 Fay，和新增 Email 是哪種改變？

兩支都從 MEMBER 的 M1 Dana、M2 Evan 開始。A 新增會員 M3 Fay；B 新增 Email，明訂可尚未提供。仍使用 relational model。

A : state 多一列

MemberId、Name 不變

M1 Dana；M2 Evan

新增 M3 Fay

B : schema 多一欄

MemberId、Name、Email

M1 Dana NULL

M2 Evan NULL

新增會員和新增可記錄的資訊，改動層次不同。

反例：候選會員名單

候選列	判斷
M1 Dana；M1 Fay	錯：不同會員共用 M1；Fay 應用已核定的 M3
M1 Dana；新會員 M4 Dana	另一位新會員可同名；M3 Fay 不變

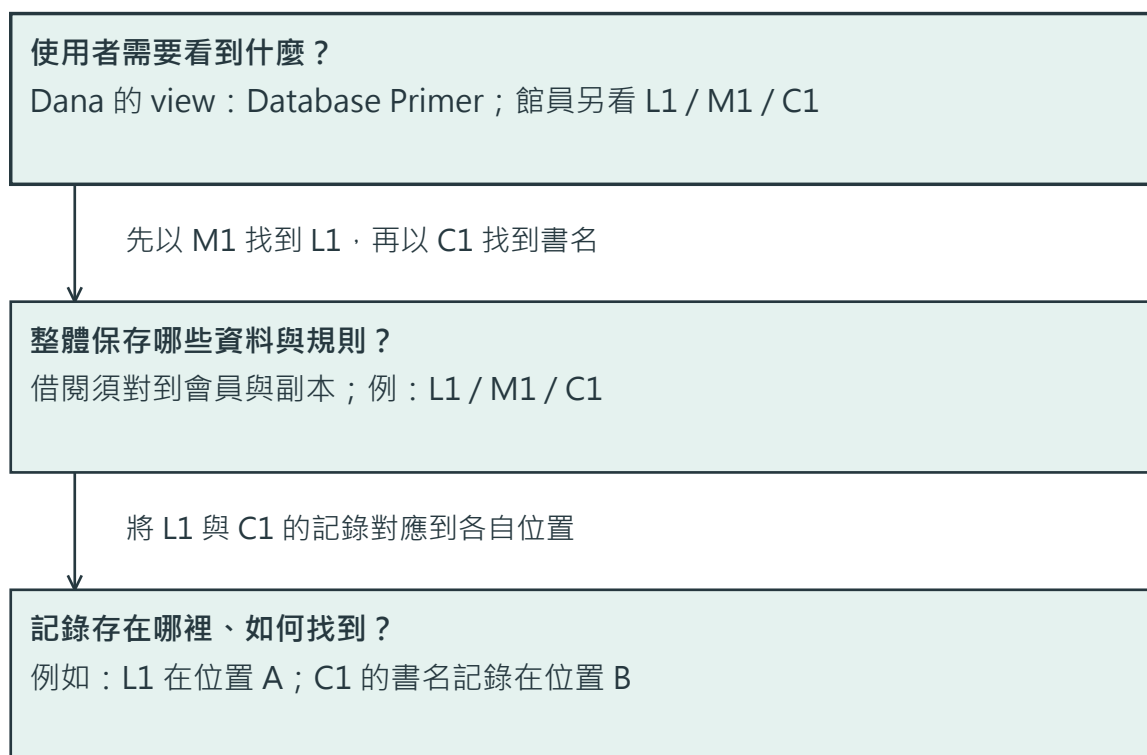
問題：同名就不能加入會員，是否符合本例？

回答：不符合。規則允許姓名重複；識別會員的是 MemberId，不是只看 Name。

本頁重點：第二個故事仍要分清 schema、state 與明訂限制。

自助畫面、借閱模型與檔案位置

Dana 只要看「我借了 Database Primer」。館員還要找到 L1 的會員與實體書編號；兩個需求都來自相同 MEMBER、COPY、LOAN。



由上到下是 external、conceptual、internal schema；層間對應稱 mapping。三框描述同一資料庫，不是三份副本；位置只是示意。

問題：若自助畫面沒有 L1，就可從整體資料刪掉 LoanId？

回答：不能由畫面省略推出可刪除。館員仍需指出是哪次借閱；保留 L1，只調整自助畫面的呈現。

本頁重點：同一事實可在不同 view 顯示不同欄位，不必建立互不一致的副本。

改索引、加 Email，原會員名單還在嗎？

查詢一直是「列出 M1 Dana、M2 Evan」。A 新增 Name 索引；B 新增允許空白的 Email；兩支獨立，姓名與編號不變。

A : physical 變動

多了 Name 索引

MEMBER 欄位與資料不變

結果：M1 Dana；M2 Evan

B : logical 變動

MEMBER 多了 Email 欄

view 仍只取編號、姓名

結果：M1 Dana；M2 Evan

A 改存取結構；B 改整體欄位後維持原 view。兩種隔離處理的層次不同。

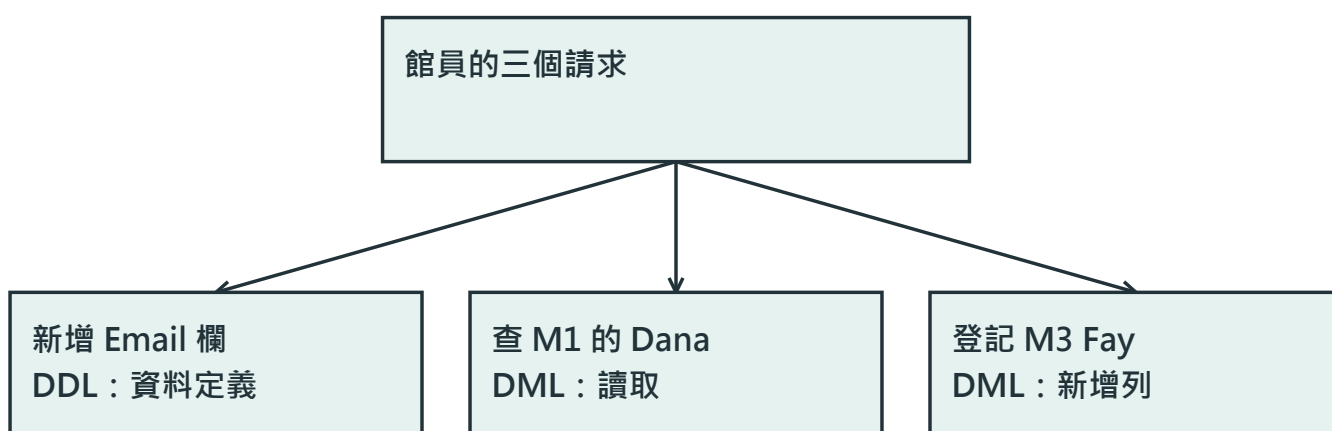
問題：若 B 改成刪掉 Name，還能承諾顯示 Dana、Evan？

回答：不能。本例沒有其他地方保存姓名；修正是保留 Name，再增加 Email，而不是刪去原事實。

本頁重點：能否保留原需求，要檢查必要資料與對應，不能只看操作名稱。

會員表單的新增，不是新增一種欄位

館員操作 MEMBER 表單。Email 新欄位尚未加入；目前仍為 M1 Dana、M2 Evan。三個請求都從這個起點獨立判斷。



箭頭表示用途分類，不表示三個請求一定依次執行。

誤判與具體修正

誤判	修正後可見結果
登記 Fay 是加新欄位	仍是 MemberId、Name，只多 M3 Fay
查 Dana 必須先新增 Dana	Dana 已在 M1；讀取即可

問題：只要點按鈕，就不需要 DDL / DML 的區分？

回答：仍要區分。介面把請求交給程式處理，但改定義與改資料會造成不同結果。

本頁重點：同樣是按表單，可能改定義、讀取資料或新增資料。

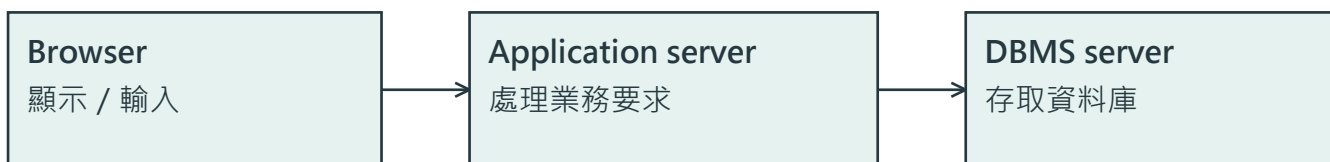
館員直連，還是經應用伺服器？

A：館員應用程式直接連 DBMS。B：會員用瀏覽器，經應用伺服器檢查身分後查借閱。兩者都用原本 MEMBER、COPY、LOAN。

A：應用程式直接請 DBMS 查資料



B：瀏覽器先把請求交給應用伺服器



A 的用戶端、DBMS 兩層稱 two-tier；B 多一層應用伺服器，稱 three-tier。箭頭是請求路徑；軟體職責不等於電腦台數。

問題：把三種軟體裝在同一台電腦，three-schema 就消失？

回答：不會。部署台數與資料描述層次是不同問題；也不能把這張設計圖說成本課已實際部署的系統。

本頁重點：Tier 看職責與溝通路徑；schema 層次看如何描述同一資料庫。